Austin Kim

Custom RISC-V CPU Project (Vivado)

Abstract

This custom RISC-V CPU implements 11-instructions based on a derivation of the RISC-V (32-bit) CPU architecture called SCU ISA. The SCU ISA architecture will use word addressing.

For my architecture, I decided not to implement the Forwarding and Hazard Detection Unit, and instead opted to hardcode NOP instruction into my assembly code.

The CPU is using the 5-stage pipelined architecture:

1. IF: Instruction Fetch
2. ID: Instruction Decode / Register Read
3. EX: Execute
4. MEM: Data Memory
5. WB: Write-Back

And I are using 4 pipeline buffers:

1. IF/ID: 64 bits
2. ID/EX: 145 bits
3. EX/MEM: 144 bits
4. MEM/WB: 105 bits

The buffer bit-widths are calculated in Table 5 of my Appendix section.

I achieved adding these NOP instructions by assessing the code for read-after-write flow dependencies.

For my simulation I chose to test the a stencil code **`CSEN122-F25-SCU_ISA_Assembly_1d_Stencil.txt`**.

I decided to write a JavaScript Program (Code 2) that converts the asm code into binary so I can easily paste it into **`inst_mem.v`**.

CPU Design Description:

Stage 1: Instruction Fetch (IF)

The PC uses a 32-bit register and is updated every cycle. A branch or jump address could also influence the changes of the PC address.

This stage has the following components:

- Instruction Memory
- PC
- adder: computing PC + 1 (word-addressing)
- mux: (logically implemented using conditionals) two connected together.
  Each mux choosing betIen:
  1. Signal: PCSrc; 0 - rs_data_EX; 1 - rs_data_MEM;
  2. Signal: J ; 0 – (PC + 1) ; 1 – 1st mux

PCSrc is comprised of go_branch and Jump (J) signal.

Refer to Stage 4: Data Memory (MEM) for a description of go_branch.

<u>Stage 2: Instruction Decode (ID)</u>
This stage contains the following components:

- Register File
- Immediate Generator
- Control Unit

Register File
The register file contains 64 register by 32 bits. The design uses synchronous write and read on the positive edge.

Immediate Generator
The immediate field (10-bit signed) is extracted from the instruction [31-22] and sign-extended to 32-bits. This is needed for instructions: LD, ST, SVPC, INC, as Ill as for computing branch and jump targets.

Control Unit
The control unit interprets instruction opcode bits [3:0] and produces WB, M, and EX signals. Refer to Table 4 : Signal Size for more information.

<u>Execute (EX) Stage 3</u>
This stage contains the following component:

- ALU
- adder
- ALU Control
- mux

ALU and ALU Control
The ALU implements the operands listed in Table 2 : ALU Truth Table.
The ALUOp (2-bit) signal from EX feeds into the ALU control unit which provides the correct (add, sub, neg) signals to the ALU operation.

Branch and Jump Addressing
The adder computes PC + y for SVPC.
For branching, the address stored in rs_data is referred by the MEM stage.
For jumping, I use the rs_data that is referred in the MEM stage.

<u>Stage 4: Data Memory (MEM)</u>
This stage contains the following components:

- Data Memory
- AND and OR gates

Data Memory takes two signals:

- MemRead: $x\text{rs} \leftarrow \text{MEM}[x\text{rs} + y]$
- MemWrite: $\text{MEM}[x\text{rs} + y] \leftarrow \text{rt}$

The AND and OR gates are used to calculate PCSrc, simply:

$$\text{PCSrc} = \text{N_MEM} \ \& \ \text{N_EX} \mid \text{Z_MEM} \ \& \ \text{Z_EX}$$

Stage 5: Write-Back (WB)

This stage contains the following components:

- mux: two that are connected to each other

These mux are connected and controlled as follows:

1. Signal: MemToReg; 0 – OUT ; 1 – read_data
2. Signal: SVPC ; 0 – 1st mux; 1 – PC_imm

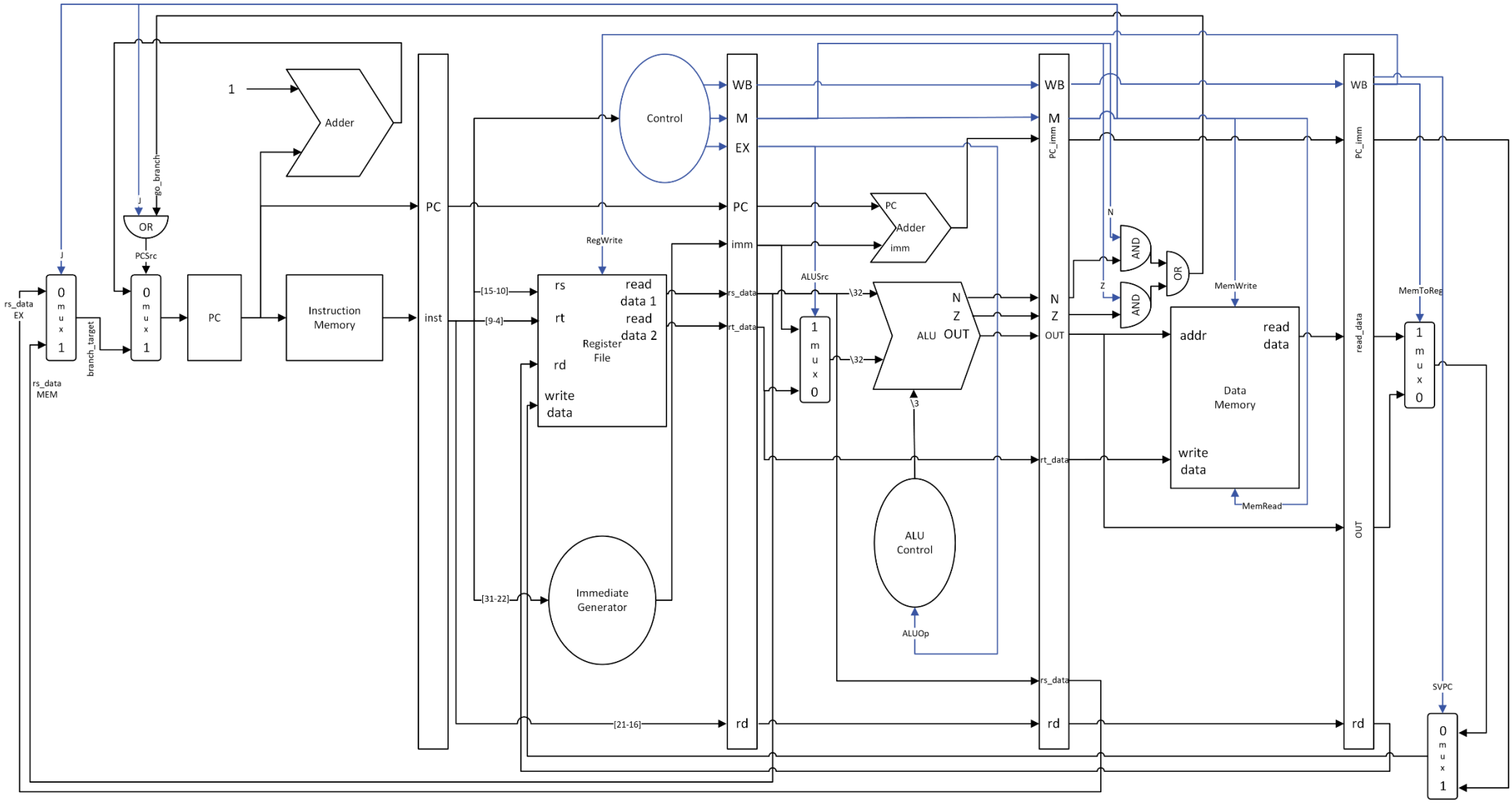
OUT is the ALU result.
read_data is from the data memory.
PC_imm is simply PC + immediate value.

Instruction Support

Instruction	Syntax	y	rd	rs	rt	Opcode	Function
No operation	NOP	x	x	x	x	0000	
Save PC	SVPC rd, y	y	rd	x	x	1111	
Load	LD rd, rs, y	y	rd	rs	x	1110	
Store	ST rt, rs, y	y	x	rs	rt	0011	
Add	ADD rd, rs, rt	x	rd	rs	rt	0100	
Increment	INC rd, rs, y	y	rd	rs	x	0101	
Negate	NEG rd, rs	x	rd	rs	x	0110	
Subtract	SUB rd, rs, rt	x	rd	rs	rt	0111	
Jump	J rs	x	x	rs	x	1000	
Branch if zero	BRZ rs	x	x	rs	x	1001	
Branch if negative	BRN rs	x	x	rs	x	1010	

Special instruction SCU ISA supports are NOP, SVPC, BRN, and BRZ.
NOP will deactivate all control signal and is used as a bubble.
SVPC saves the return location when branching. It can also be used to refer specific functions.
BRN will branch if the previous instruction calculated a negative result in the ALU.
BRZ will branch if the previous instruction calculated a zero result in the

Datapath Diagram



Vivado Simulation Results

```
Starting Simulation.
--------------------------------------------------
Checking A array is correctly initialized
--------------------------------------------------
[PASS] Index x2: Correct (3)
[PASS] Index x3: Correct (1)
[PASS] Index x4: Correct (5)
[PASS] Index x5: Correct (7)
[PASS] Index x6: Correct (2)
[PASS] Index x7: Correct (9)
[PASS] Index x8: Correct (8)
--------------------------------------------------
Checking SVPC Data Setup (<00>-<07>)
--------------------------------------------------
[PASS] <00> x40: Correct immediate (19)
[PASS] <01> x41: Correct immediate (70)
[PASS] <02> x42: Correct immediate (50)
[PASS] <03> x43: Correct immediate (56)
[PASS] <04> x44: Correct immediate (63)
[PASS] <05> x45: Correct immediate (30)
[PASS] <06> x46: Correct immediate (34)
[PASS] <07> x47: Correct immediate (38)
--------------------------------------------------
Checking Final Result (Data Mem - Indices 71-77)
--------------------------------------------------
[PASS] Index x71: Correct (3)
[PASS] Index x72: Correct (3)
[PASS] Index x73: Correct (5)
[PASS] Index x74: Correct (5)
[PASS] Index x75: Correct (7)
[PASS] Index x76: Correct (8)
[PASS] Index x77: Correct (8)
--------------------------------------------------
```

Figure 1: Simulation Result

Vivado Simulation Report

In Figure 1, the simulation code (Code 1) resulted in the correct initialization and results.

Performance Analysis

The Median 1D Stencil code that was edited to include NOP instruction and the total time to complete instruction execution was **2510 ns** with a clock of **100 MHz**.

$$f = \frac{1}{T} \text{ where } T = 10 \text{ ns}$$

$$= \frac{1}{10 \cdot 10^{-9}} = 100 \cdot 10^{6} \text{ hz} \equiv 100 \text{ Mhz}$$

This also means that it <u>took 251 instructions to complete</u> with an array size of $\boldsymbol{n = 7}$.

$$\text{CPU time} = \text{Cycle Time} \cdot \text{Cycle Count}$$

$$\text{Cycle Count} = \frac{\text{CPU time}}{\text{Cycle Time}} = \frac{2510 \cdot 10^{-9}}{10 \cdot 10^{-9}} = 251 \text{ cycles}$$

Since the program executed 251 instructions for an input array of $n = 7$, the CPI estimate is 1.0.

$$\text{CPI} \approx \frac{\text{Total Cycles}}{\text{Instruction Count}} = \frac{251}{251} \approx \mathbf{1.0} \left[\frac{\text{cycle}}{\text{instruction}}\right]$$

Additionally, the inserted NOPs correspond to the added instruction.

To verify this I can estimate the number of looped and non-looped instruction.

$$\text{Instruction Count}(n) = I_{\text{non-loop}} + I_{\text{loop}} \cdot n$$

$$\text{Cycle Count}(n) \approx I_{\text{non-loop}} + I_{\text{loop}} \cdot n$$

For simulation with $n = 7$

$$I_{\text{loop}} \approx \frac{251 - I_{\text{non-loop}}}{7} \approx \frac{251 - 36}{7} \approx \textbf{30 instructions per iteration}$$

This verification lines up with
Verification: (Final Result) and Waveform Example 1.

I can further reduce the number of cycles by implementing a forwarding unit or hazard detection.

Appendix

Instruction Truth Table:

Inst.	OP	ALUSrc	MemToReg	RegWrite	MemRead	MemWrite	ALUOp (2)	SVPC	Z	N	J
NOP	0000	0	0	0	0	0	xx	0	0	0	0
SVPC	1111	1	x	1	0	0	00	1	0	0	0
LD	1110	1	1	1	1	0	00	0	0	0	0
ST	0011	1	0	0	0	1	00	0	0	0	0
ADD	0100	0	0	1	0	0	00	0	0	0	0
INC	0101	1	0	1	0	0	00	0	0	0	0
NEG	0110	0	0	1	0	0	10	0	0	0	0
SUB	0111	0	0	1	0	0	01	0	0	0	0
J	1000	0	x	0	0	0	xx	x	x	x	1
BRZ	1001	x	x	0	0	0	xx	x	1	0	0
BRN	1010	x	x	0	0	0	xx	x	0	1	0

Table 1: Instruction Truth Table

ALU Truth Table

ALUOp	Instruction [3:0]	ALU Control Output (3 bits)	Operation
00	1111,1110,0100,0101,0011	000	Add
01	0111,1001	101	Subtract
10	0110	110	Negate
11			Pass A

Table 2 : ALU Truth Table

ALU Control Output Signals: ADD, NEG, SUB

ALU Truth Table:

ADD	NEG	SUB	OUT	Operation
0	0	0	A + B	Add
0	1	0	Don't Care	No Operation
1	0	1	A – B	Subtract
1	1	0	-A	2's Complement
1	1	1	A	Pass A

Table 3: ALU Unit Truth Table

Signals:

WB	(3 bits total):	SVPC, MemToReg, RegWrite
IF/EX	(5 bits total):	J, N, Z, MemRead, MemWrite
EX/MEM	(3 bits total):	ALU Op (2 bits), ALUSrc

Table 4 : Signal Size

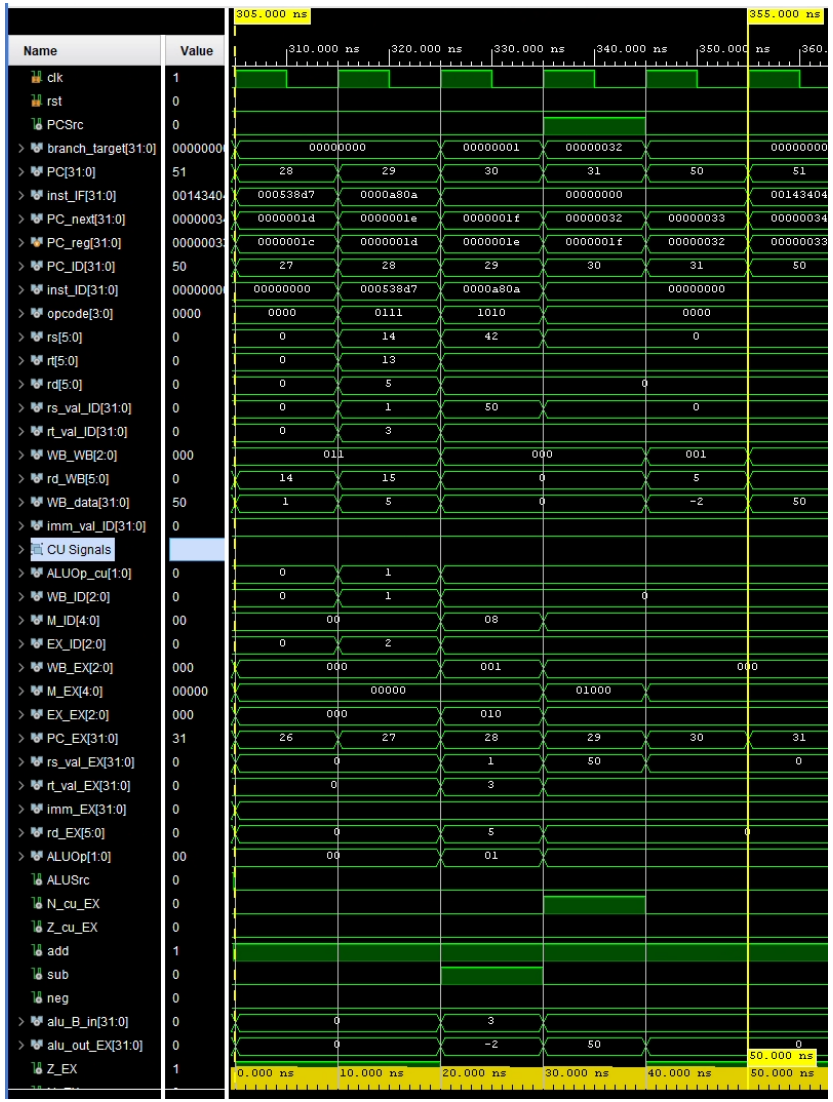
Buffer Size:

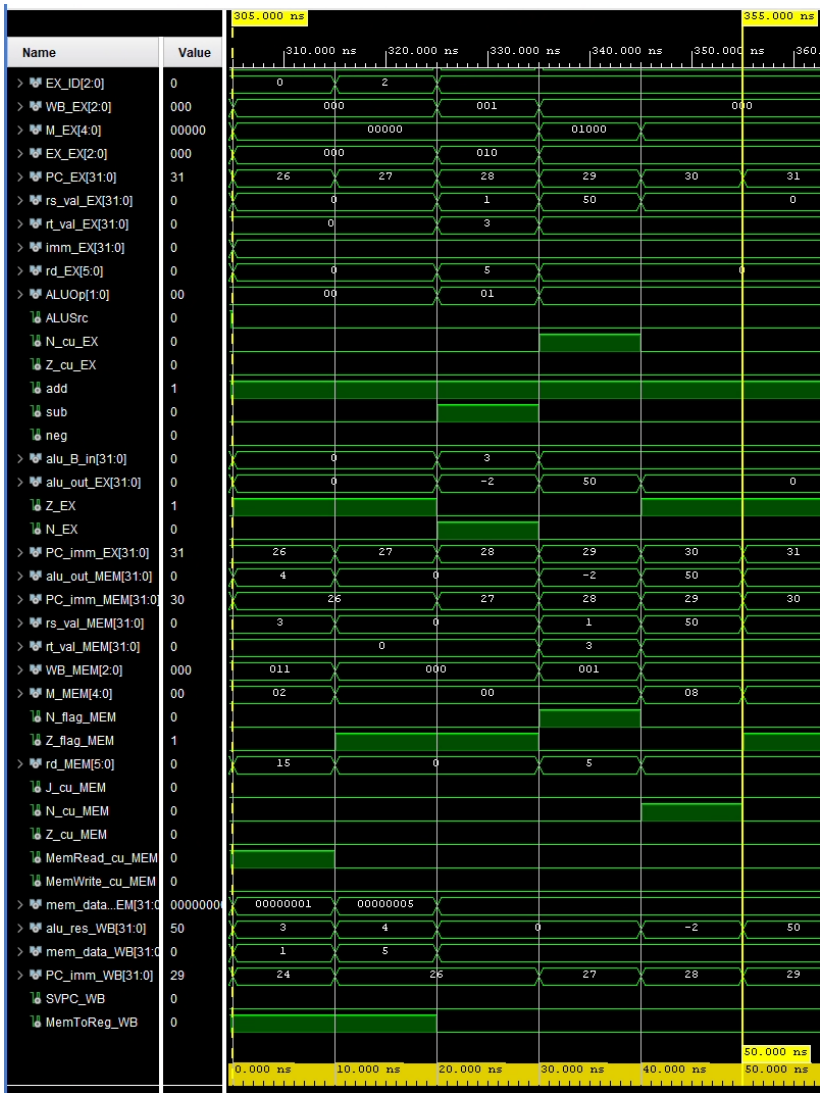
ID/IF	(64 bits total):	PC(32), inst(32)
IF/EX	(145 bits total):	WB(3), M(5), EX(3), PC(32), imm(32), rs_data(32), rt_data(32), rd(6)
EX/MEM	(144 bits total):	WB(3), M(5), PC_imm(32), N(1), Z(1), OUT(32), rs_data(32), rt_data(32), rd(6)
MEM/WB	(105 bits total):	WB(3), PC_imm(32), read_data(32), OUT(32), rd(6)

Table 5 : Buffer Size

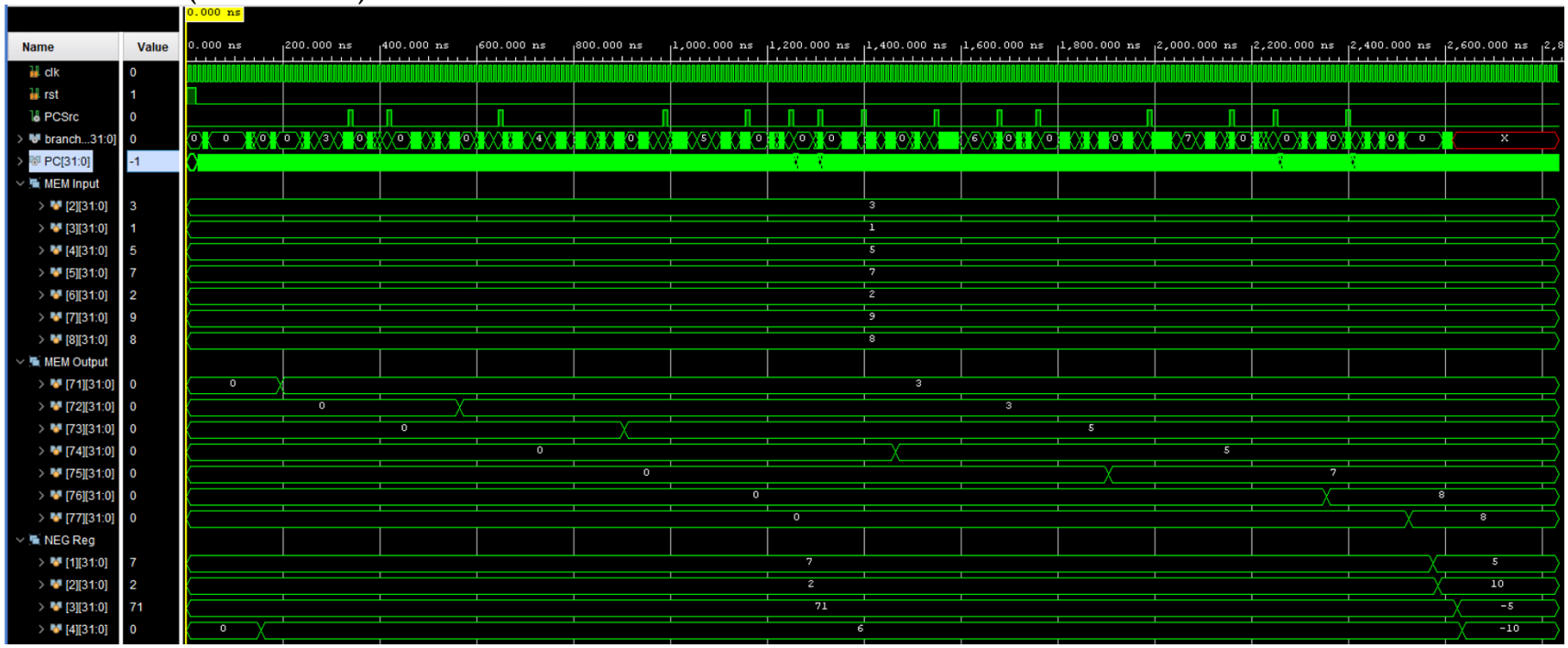
Note that buffer has CLK and RST signal, adding an additional 2 (that is not included in calculation above).

Waveform Example 1





Verification: (Final Result)



I can visually see that MEM Input was set correctly since the beginning of the program, labeled as `**MEM Input**`. As the program progresses and finds the median, I can visually see `**MEM Output**` getting set individually. I verified that this result is correct, as I set my array `**B**` to be in the data memory address `**71**`.

The median stencil code that was provided by the professor did not have NEG instruction so I tested it at the end of the median stencil code. I set registry `**1**` to 5 and registry `**2**` to 10. I then made sure that running the negative instruction would store the negative values into the registry (as observed in registry `**3**` and `**4**`).
Refer to Code 5 for more the smyce code of NEG test.

Verification: each loop is around 30 instructions, if you measure the difference betIen the `**MEM Output**` updates.
e.g., the difference betIen data memory address `**73**` and `**72**` is around 320ns which is 32 cycles and with CPI of 1.0, that's 32 instructions.

Code 1: Simulation (tb_cpu.v)

```
`timescale 1ns / 1ps
module tb_cpu;
    reg clk;
    reg rst;

    cpu uut (
        .clk(clk),
        .rst(rst)
    );

    initial begin
        clk = 0;
        forever #5 clk = ~clk;
    end

    initial begin
        rst = 1;
        #20;
        rst = 0;
        $display("Starting Simulation.");
    end

    integer i;
    reg [6:0]  idx [0:7];
    reg [31:0] exp_val [0:7];

    initial begin
        $display("----------------------------------------------------");
        $display("Checking A array is correctly initialized         ");
        $display("----------------------------------------------------");


        exp_val[0] = 32'd3;
        exp_val[1] = 32'd1;
        exp_val[2] = 32'd5;
        exp_val[3] = 32'd7;
        exp_val[4] = 32'd2;
        exp_val[5] = 32'd9;
        exp_val[6] = 32'd8;
    end

    initial begin
        #20;
        #80;
        #50;

        $display("----------------------------------------------------");
        $display("Checking SVPC Data Setup (<00>-<07>)              ");
        $display("----------------------------------------------------");

        idx[0] = 7'd40;  exp_val[0] = 32'd19;
        idx[1] = 7'd41;  exp_val[1] = 32'd70;
        idx[2] = 7'd42;  exp_val[2] = 32'd50;
        idx[3] = 7'd43;  exp_val[3] = 32'd56;
        idx[4] = 7'd44;  exp_val[4] = 32'd63;
        idx[5] = 7'd45;  exp_val[5] = 32'd30;
        idx[6] = 7'd46;  exp_val[6] = 32'd34;
        idx[7] = 7'd47;  exp_val[7] = 32'd38;

        for (i = 0; i < 8; i = i + 1) begin
            if (uut.regfile.regs[idx[i]] === exp_val[i]) begin
                $display("[PASS] <%02d> x%0d: Correct immediate (%0d)",
                        i, idx[i], exp_val[i]);
            end else begin
                $display("[FAIL] <%02d> x%0d: Found %0d, Expected (%0d)",
                        i, idx[i], uut.regfile.regs[idx[i]], exp_val[i]);
            end
        end
    end

    initial begin
        #3500;
        idx[0] = 7'd71; exp_val[0] = 32'd3;
        idx[1] = 7'd72; exp_val[1] = 32'd3;
        idx[2] = 7'd73; exp_val[2] = 32'd5;
        idx[3] = 7'd74; exp_val[3] = 32'd5;
        idx[4] = 7'd75; exp_val[4] = 32'd7;
        idx[5] = 7'd76; exp_val[5] = 32'd8;
        idx[6] = 7'd77; exp_val[6] = 32'd8;

        $display("----------------------------------------------------");
        $display("Checking Final Result (Data Mem - Indices 71-77)  ");
        $display("----------------------------------------------------");
        for (i = 0; i < 7; i = i + 1) begin
            if (uut.dm.mem[idx[i]] === exp_val[i]) begin
                $display("[PASS] Index x%0d: Correct (%0d)",
                        idx[i], exp_val[i]);
            end else begin
                $display("[FAIL] Index x%0d: Found %0d, Expected (%0d)",
                        idx[i], uut.dm.mem[idx[i]], exp_val[i]);
            end
        end

        $display("----------------------------------------------------");
        $finish;
    end

endmodule
```

Code 2: JavaScript Convert Code (convert.js)

```javascript
const COMMENT_DELIM = "//";
const fs = require('fs');
const readline = require('readline').createInterface({
    input: process.stdin,
    output: process.stdout
});
readline.question('Enter the path to the assembly file: ', filePath => {
    fs.readFile(filePath, 'utf8', (err, data) => {
        if (err) {
            console.error('Error reading file:', err);
            readline.close();
            return;
        }
        const lines = data.split('\n');
        let outputLines = [];
        for (let line of lines) {
            if (line.includes(COMMENT_DELIM))
                line = line.split(COMMENT_DELIM)[0].trim();
            if (line === '') continue;
            line = line.replace(/\s+/g, ' ');
            const match = line.match(/<(\d+)>:\s+([A-Z]+)(?:\s+([x0-9-,\s]+))?/);
            if (match) {
                const address = parseInt(match[1]);
                const opcode = match[2];
                let operands = undefined;
                if (match[3])
                    operands = match[3].split(',').map(op => op.trim());
                const binaryInstruction = convertToBinary(opcode, operands);
                outputLines.push(`memory[${address}] = 32'b${binaryInstruction};`);
            }
        }
        const outputFilePath = filePath.replace('.asm', '_convert.txt');
        fs.writeFile(outputFilePath, outputLines.join('\n'), err => {
            if (err) console.error('Error writing file:', err);
            else console.log('File converted successfully:', outputFilePath);
            readline.close();
        });
    });
});
function convertToBinary(opcode, operands) {
    const opcodeMap = {
        'NOP': '0000',
        'SVPC': '1111',
        'LD': '1110',
        'ST': '0011',
        'ADD': '0100',
        'INC': '0101',
        'NEG': '0110',
        'SUB': '0111',
        'J': '1000',
        'BRZ': '1001',
        'BRN': '1010'
    };
    let Y = '0000000000';
    let RD = '000000';
    let RS = '000000';
    let RT = '000000';
    let OPCODE = opcodeMap[opcode];
    switch (opcode) {
        case 'NOP':
            break;
        case 'SVPC':
            RD = registerToBinary(operands[0]);
            Y = immediateToBinary(operands[1], 10);
            break;
        case 'LD':
            RD = registerToBinary(operands[0]);
            RS = registerToBinary(operands[1]);
            Y = immediateToBinary(operands[2], 10);
            break;
        case 'ST':
            RT = registerToBinary(operands[0]);
            RS = registerToBinary(operands[1]);
            Y = immediateToBinary(operands[2], 10);
            break;
        case 'ADD':
            RD = registerToBinary(operands[0]);
            RS = registerToBinary(operands[1]);
            RT = registerToBinary(operands[2]);
            break;
        case 'INC':
            RD = registerToBinary(operands[0]);
            RS = registerToBinary(operands[1]);
            Y = immediateToBinary(operands[2], 10);
            break;
        case 'NEG':
            RD = registerToBinary(operands[0]);
            RS = registerToBinary(operands[1]);
            break;
        case 'SUB':
            RD = registerToBinary(operands[0]);
            RS = registerToBinary(operands[1]);
            RT = registerToBinary(operands[2]);
            break;
        case 'J':
            RS = registerToBinary(operands[0])
            break;
        case 'BRZ':
            RS = registerToBinary(operands[0])
            break;
        case 'BRN':
            RS = registerToBinary(operands[0])
            break;
        default:
            console.error('Unknown opcode:', opcode);
            break;
    }
    return `${Y}_${RD}_${RS}_${RT}_${OPCODE}`;
}

function registerToBinary(register) {
    const regNum = parseInt(register.slice(1));
    return regNum.toString(2).padStart(6, '0');
}

function immediateToBinary(immediate, bits) {
    let immNum = parseInt(immediate);
    if (immNum < 0) {
        immNum = (1 << bits) + immNum;
    }
    return immNum.toString(2).padStart(bits, '0');
}
```

Code 3: ASM Code (Median_stencil.asm)

```
<00>:  SVPC   x40, 19        // save addr of LOOP_BODY (y = 19-0)
<01>:  SVPC   x41, 69        // save addr of EPILOGUE (y = 70-1)
<02>:  SVPC   x42, 48        // save addr of SWAP_1 (y = 50-2)
<03>:  SVPC   x43, 53        // save addr of SWAP_2 (y = 56-3)
<04>:  SVPC   x44, 59        // save addr of SWAP_3 (y = 63-4)
<05>:  SVPC   x45, 25        // return addr from SWAP_1 (y = 30-5)
<06>:  SVPC   x46, 28        // return addr from SWAP_2 (y = 34-6)
<07>:  SVPC   x47, 31        // return addr from SWAP_3 (y = 38-7)
<08>:  INC    x4, x1, -1     // save n-1 to x4
<09>:  LD     x20, x2, 0     // load A[0] to x20;  y=0, rd=x10, rs=x2, rt=NA
<10>:  NOP
<11>:  NOP
<12>:  NOP
<13>:  ST     x20, x3, 0     // store x20 to B[0]; y=0, rd=NA, rs=x3, rt=x10
<14>:  SUB    x0, x0, x0     // making x0 as 0
<15>:  INC    x10, x0, 1     // set x10 (for variable i) as 1
<16>:  NOP
<17>:  NOP
<18>:  NOP
// LOOP_BODY:
<19>:  ADD    x11, x2, x10   // addr of A[i]:x11=&A[0]+i
<20>:  ADD    x12, x3, x10   // addr of B[i]:x12=&B[0]+i
<21>:  NOP
<22>:  NOP
<23>:  LD     x13, x11, -1   // load A[i-1] to x13
<24>:  LD     x14, x11, 0    // load A[i] to x14
<25>:  LD     x15, x11, 1    // load A[i+1] to x15
<26>:  NOP
<27>:  NOP
<28>:  SUB    x5, x14, x13   // x5 = x14 - x13 = 1 - 3 = -2 (correct output)
<29>:  BRN    x42            // SWAP_1 if x13 > x14 : x42 (got address 34)
<30>:  NOP                   // SWAP_1 returns here to avoid hazard
<31>:  NOP
<32>:  SUB    x5, x15, x14
<33>:  BRN    x43            // SWAP_2 if x15 > x14
<34>:  NOP                   // SWAP_2 returns here to avoid hazard
<35>:  NOP
<36>:  SUB    x5, x14, x13
<37>:  BRN    x44            // SWAP_3 if x13 > x14
                             // after this, x14 holds the median of
                             // the three elements
<38>:  NOP                   // SWAP_3 returns here to avoid hazard
<39>:  NOP
<40>:  ST     x14, x12, 0    // save x14 to B[i]
<41>:  INC    x10, x10, 1    // i = i+1
<42>:  NOP
<43>:  NOP
<44>:  NOP
<45>:  SUB    x5, x10, x4    // to compare i and n-1
<46>:  BRZ    x41            // goto epilogue if epilogue if i == n-1
<47>:  NOP
<48>:  NOP
<49>:  J      x40            // repeat LOOP_BODY
// SWAP_1 (swap x13 and x14)
<50>:  NOP
<51>:  ADD    x20, x13, x0   // x13 = 3
<52>:  ADD    x13, x14, x0   // x14 = 1
<53>:  NOP
<54>:  J      x45
<55>:  ADD    x14, x20, x0   // x14 = 3
// SWAP_2 (swap x14 and x15)
<56>:  NOP
<57>:  NOP
<58>:  ADD    x20, x14, x0
<59>:  ADD    x14, x15, x0
<60>:  NOP
<61>:  J      x46
<62>:  ADD    x15, x20, x0
// SWAP_3 (swap x13 and x14)
<63>:  NOP
<64>:  NOP
<65>:  ADD    x20, x13, x0
<66>:  ADD    x13, x14, x0
<67>:  NOP
<68>:  J      x47
<69>:  ADD    x14, x20, x0
// EPILOGUE
<70>:  ADD    x11, x2, x10
<71>:  ADD    x12, x3, x10
<72>:  NOP
<73>:  NOP
<74>:  LD     x20, x11, 0
<75>:  NOP
<76>:  NOP
<77>:  NOP
<78>:  ST     x20, x12, 0
<79>:  NOP
<80>:  NOP
<81>:  SUB    x0, x0, x0
<82>:  INC    x1, x0, 5
<83>:  INC    x2, x0, 10
<84>:  NEG    x3, x1
<85>:  NEG    x4, x2
// For next instruction (technically not needed but added for safety)
<86>:  NOP
<87>:  NOP
<88>:  NOP
```

Code 4: ASM Median Stencil Converted (Binary Program)

```
memory[0] = 32'b0000010011_101000_000000_000000_1111;
memory[1] = 32'b0001000101_101001_000000_000000_1111;
memory[2] = 32'b0000110000_101010_000000_000000_1111;
memory[3] = 32'b0000110101_101011_000000_000000_1111;
memory[4] = 32'b0000111011_101100_000000_000000_1111;
memory[5] = 32'b0000011001_101101_000000_000000_1111;
memory[6] = 32'b0000011100_101110_000000_000000_1111;
memory[7] = 32'b0000011111_101111_000000_000000_1111;
memory[8] = 32'b1111111111_000100_000001_000000_0101;
memory[9] = 32'b0000000000_010100_000010_000000_1110;
memory[10] = 32'b0000000000_000000_000000_000000_0000;
memory[11] = 32'b0000000000_000000_000000_000000_0000;
memory[12] = 32'b0000000000_000000_000000_000000_0000;
memory[13] = 32'b0000000000_000000_000011_010100_0011;
memory[14] = 32'b0000000000_000000_000000_000000_0111;
memory[15] = 32'b0000000001_001010_000000_000000_0101;
memory[16] = 32'b0000000000_000000_000000_000000_0000;
memory[17] = 32'b0000000000_000000_000000_000000_0000;
memory[18] = 32'b0000000000_000000_000000_000000_0000;
memory[19] = 32'b0000000000_001011_000010_001010_0100;
memory[20] = 32'b0000000000_001100_000011_001010_0100;
memory[21] = 32'b0000000000_000000_000000_000000_0000;
memory[22] = 32'b0000000000_000000_000000_000000_0000;
memory[23] = 32'b1111111111_001101_001011_000000_1110;
memory[24] = 32'b0000000000_001110_001011_000000_1110;
memory[25] = 32'b0000000001_001111_001011_000000_1110;
memory[26] = 32'b0000000000_000000_000000_000000_0000;
memory[27] = 32'b0000000000_000000_000000_000000_0000;
memory[28] = 32'b0000000000_000101_001110_001101_0111;
memory[29] = 32'b0000000000_000000_101010_000000_1010;
memory[30] = 32'b0000000000_000000_000000_000000_0000;
memory[31] = 32'b0000000000_000000_000000_000000_0000;
memory[32] = 32'b0000000000_000101_001111_001110_0111;
memory[33] = 32'b0000000000_000000_101011_000000_1010;
memory[34] = 32'b0000000000_000000_000000_000000_0000;
memory[35] = 32'b0000000000_000000_000000_000000_0000;
memory[36] = 32'b0000000000_000101_001110_001101_0111;
memory[37] = 32'b0000000000_000000_101100_000000_1010;
memory[38] = 32'b0000000000_000000_000000_000000_0000;
memory[39] = 32'b0000000000_000000_000000_000000_0000;
memory[40] = 32'b0000000000_000000_001100_001110_0011;
memory[41] = 32'b0000000001_001010_001010_000000_0101;
memory[42] = 32'b0000000000_000000_000000_000000_0000;
memory[43] = 32'b0000000000_000000_000000_000000_0000;
memory[44] = 32'b0000000000_000000_000000_000000_0000;
memory[45] = 32'b0000000000_000101_001010_000100_0111;
memory[46] = 32'b0000000000_000000_101001_000000_1001;
memory[47] = 32'b0000000000_000000_000000_000000_0000;
memory[48] = 32'b0000000000_000000_000000_000000_0000;
memory[49] = 32'b0000000000_000000_101000_000000_1000;
memory[50] = 32'b0000000000_000000_000000_000000_0000;
memory[51] = 32'b0000000000_010100_001101_000000_0100;
memory[52] = 32'b0000000000_001101_001110_000000_0100;
memory[53] = 32'b0000000000_000000_000000_000000_0000;
memory[54] = 32'b0000000000_000000_101101_000000_1000;
memory[55] = 32'b0000000000_001110_010100_000000_0100;
memory[56] = 32'b0000000000_000000_000000_000000_0000;
memory[57] = 32'b0000000000_000000_000000_000000_0000;
memory[58] = 32'b0000000000_010100_001110_000000_0100;
memory[59] = 32'b0000000000_001110_001111_000000_0100;
memory[60] = 32'b0000000000_000000_000000_000000_0000;
memory[61] = 32'b0000000000_000000_101110_000000_1000;
memory[62] = 32'b0000000000_001111_010100_000000_0100;
memory[63] = 32'b0000000000_000000_000000_000000_0000;
memory[64] = 32'b0000000000_000000_000000_000000_0000;
memory[65] = 32'b0000000000_010100_001101_000000_0100;
memory[66] = 32'b0000000000_001101_001110_000000_0100;
memory[67] = 32'b0000000000_000000_000000_000000_0000;
memory[68] = 32'b0000000000_000000_101111_000000_1000;
memory[69] = 32'b0000000000_001110_010100_000000_0100;
memory[70] = 32'b0000000000_001011_000010_001010_0100;
memory[71] = 32'b0000000000_001100_000011_001010_0100;
memory[72] = 32'b0000000000_000000_000000_000000_0000;
memory[73] = 32'b0000000000_000000_000000_000000_0000;
memory[74] = 32'b0000000000_010100_001011_000000_1110;
memory[75] = 32'b0000000000_000000_000000_000000_0000;
memory[76] = 32'b0000000000_000000_000000_000000_0000;
memory[77] = 32'b0000000000_000000_000000_000000_0000;
memory[78] = 32'b0000000000_000000_001100_010100_0011;
memory[79] = 32'b0000000000_000000_000000_000000_0000;
memory[80] = 32'b0000000000_000000_000000_000000_0000;
memory[81] = 32'b0000000000_000000_000000_000000_0111;
memory[82] = 32'b0000000101_000001_000000_000000_0101;
memory[83] = 32'b0000001010_000010_000000_000000_0101;
memory[84] = 32'b0000000000_000011_000001_000000_0110;
memory[85] = 32'b0000000000_000100_000010_000000_0110;
memory[86] = 32'b0000000000_000000_000000_000000_0000;
memory[87] = 32'b0000000000_000000_000000_000000_0000;
memory[88] = 32'b0000000000_000000_000000_000000_0000;
```

Code 5: NEG Instruction (Continuation of the previous median program)

```
<79>:    NOP
<80>:    NOP
<81>:    SUB     x0, x0, x0
<82>:    INC     x1, x0, 5
<83>:    INC     x2, x0, 10
<84>:    NOP
<85>:    NOP
<86>:    NOP
<87>:    NEG     x3, x1
<88>:    NEG     x4, x2
```

Code 6 : NEG Binary

```
memory[79] = 32'b0000000000_000000_000000_000000_0000;
memory[80] = 32'b0000000000_000000_000000_000000_0000;
memory[81] = 32'b0000000000_000000_000000_000000_0111;
memory[82] = 32'b0000000101_000001_000000_000000_0101;
memory[83] = 32'b0000001010_000010_000000_000000_0101;
memory[84] = 32'b0000000000_000000_000000_000000_0000;
memory[85] = 32'b0000000000_000000_000000_000000_0000;
memory[86] = 32'b0000000000_000000_000000_000000_0000;
memory[87] = 32'b0000000000_000011_000001_000000_0110;
memory[88] = 32'b0000000000_000100_000010_000000_0110;
```